1. SPECIFICATION.md (v8.1)

```
# `.uni` (Universal Neural Interface / Universal Node Identity) Format Specification
**Version:** 8.1.0 (Universal Reality-Verified & Holo-Stream Baseline)
**Status:** Open Standard Specification & Prior Art Baseline
**License:** Uni-Symbiosis Public License v1.0 (USPL-1.0) / CC-BY-4.0

---

## 1. Architectural Overview & Core Principles

Traditional document, media, and serialization formats (PDF, DOCX, HTML, JSON, raw video streams) are designed for human visual rendering or verbose text parsing, forcing AI systems, medical pods, and holographic edge processors to waste computational and network resources translating character or pixel streams into multidimensional vectors.

The **`.uni`** standard eliminates this bottleneck by operating as a binary-serialized, non-executable semantic, telemetry, and holographic wavefront container engineered for direct tensor operations, granular cryptographic disclosure, and automated cryptographic notarization.

### Fundamental Design Guarantees
1. **Zero-Parsing Inference:** Native storage of bit-exact IEEE 754 floating-point embeddings (`float32` tensors), binary holographic phase arrays, and atomic Knowledge Graph triplets (`[Subject] -> [Predicate] -> [Object]`) enables immediate vector similarity comparison, graph traversal, and optical deflection rendering without text parsing.
2. **Zero-Execution Environment:** The `.uni` container is strictly a passive data structure incapable of housing executable binary opcodes, macros, or arbitrary scripts. Incoming streams are validated at the buffer boundary via strict Type-Length-Value (TLV) schema enforcement (`sterile_ingest`), reducing the code-execution attack surface to zero.
3. **Dual-Layer Genesis Envelope (Backward Compatibility):** A single `.uni` container simultaneously supports modern AI/optical accelerators (via binary tensor and wavefront nodes) and legacy systems (via a human-readable Markdown/UTF-8 node or deterministic transcoding into DICOM, HL7 FHIR, and PDF).
4. **Granular Per-Node Encryption & Selective Disclosure:** Each internal node is encrypted independently using `AES-256-GCM` with keys derived via `RFC 5869 HKDF-SHA256` and bound via a SHA-256 Merkle Tree. The container owner can disclose and mathematically prove the authenticity of a single node (e.g., for an environmental LEED/BREEAM
```

auditor or medical specialist) via an `Ed25519`-anchored `Merkle Branch Proof` without exposing any other nodes in the file.
5. **Cryptographic Notary & Anti-Coercion Layer:** Built-in binary `Ed25519` digital signatures, a 64-bit microsecond timestamp, a two-phase cooling-off timelock (Δt), and dual-key steganographic duress detection (`Primary Key` vs. `Duress Key`) replace paper notarization while protecting the human host against coercion and impulsive affect.

---

2. Binary Container Topology & 32-Byte $O(1)$ Header (`UniHeader`)

Every `.uni` v8.1 file begins with a fixed **32-byte AVX2-aligned header** encoded in Little-Endian byte order (`<4sHHIIQ4s4s`). Network controllers (SmartNIC/eBPF), optical data-diode buffers, and holographic perimeter frames (`Holo-Frame`) validate this header in $O(1)$ constant time prior to dynamic heap allocation.

```text
+-------------------------------------------------------------------
--+
| 0x00 - 0x03 (4B) | MAGIC_BYTES   : ASCII "UNI8" (0x55 0x4E 0x49
0x38) |
+-------------------------------------------------------------------
--+
| 0x04 - 0x05 (2B) | PROTO_FLAGS   : Version (0x0810) & Mode Bitmask
|
+-------------------------------------------------------------------
--+
| 0x06 - 0x07 (2B) | NODE_BITMAP   : 16-bit Active Node Presence Mask
|
+-------------------------------------------------------------------
--+
| 0x08 - 0x0B (4B) | PAYLOAD_LIMIT : Exact Encrypted Node Stream Size
|
+-------------------------------------------------------------------
--+
| 0x0C - 0x0F (4B) | SESSION_SALT  : 32-bit Hardware TRNG Entropy Salt
|
+-------------------------------------------------------------------
--+
| 0x10 - 0x17 (8B) | TIMESTAMP_US  : 64-bit Microsecond Epoch Counter
|
+-------------------------------------------------------------------
--+
| 0x18 - 0x1B (4B) | MERKLE_TAG    : Truncated 32-bit Data Root Anchor
|
+-------------------------------------------------------------------
```

```
--+
| 0x1C - 0x1F (4B) | HEADER_ECC    : 32-bit Blake2s Header Checksum
|
+-------------------------------------------------------------------
--+
| 0x20 - 0x20+N    | Encrypted TLV Node Stream (N = PAYLOAD_LIMIT)
|
+-------------------------------------------------------------------
--+
| Optional Tail    | Reed-Solomon RS(255,223) Parity Blocks (if
0x0002) |
+-------------------------------------------------------------------
--+
```

Header Field Definitions

- **MAGIC_BYTES (4B, char[4]):** Constant "UNI8" ("UNIM" in dedicated medical hardware profiles). Identifies the stream as a non-executable .uni container.
- **PROTO_FLAGS (2B, uint16):** Protocol version (0x0810) and hardware/transport mode bitmask:
  - 0x0001 (TAMPER_MESH_INTACT): Hardware security mesh intact.
  - 0x0002 (RS_FEC_APPENDED): Reed-Solomon RS(255, 223) Forward Error Correction parity tail appended after PAYLOAD_LIMIT.
  - 0x0004 (STREAM_DELTA_FRAME): High-frequency (60–120Hz) stream delta-frame mode; omits Node 0 Ed25519 signature overhead, relying on full-header-bound AES-256-GCM session authentication.
  - 0x0008 (SPATIAL_FRUSTUM_BROADCAST): Facade-wide broadcast mode; edge controllers crop the scene according to their local Zone 0 spatial coordinates.
- **NODE_BITMAP (2B, uint16):** Bitmask indicating which node indices (0..15) are populated in the payload.
- **PAYLOAD_LIMIT (4B, uint32):** Exact byte length of the encrypted TLV node stream (excluding the 32-byte header and optional RS-FEC parity tail). Mismatched packets are dropped immediately at the hardware boundary, neutralizing buffer-overflow and Payload DoS attacks.
- **SESSION_SALT (4B, uint32):** Random salt generated per session by a hardware True Random Number Generator (TRNG) for RFC 5869 HKDF key derivation.
- **TIMESTAMP_US (8B, uint64):** 64-bit UTC microsecond counter, eliminating the Year-2038 overflow issue, preventing replay attacks, and providing microsecond phase synchronization across modular display arrays.
- **MERKLE_TAG (4B, uint8[4]):** First 4 bytes of the 256-bit SHA-256 Merkle root covering all user data nodes (1..15).
- **HEADER_ECC (4B, uint8[4]):** 4-byte Blake2s-32 digest computed over bytes 0x00..0x1B, verified in constant time.

3. Universal Node Types (UniNodeType Specification)

Following the 32-byte header, the payload consists of a sequence of cryptographically isolated nodes (indices 0..15). Each node is prefixed by a **7-byte binary descriptor** (<HBI: node_id as uint16, node_type as uint8, plaintext_byte_length as uint32), followed by a **12-byte AES-GCM Initialization Vector (IV)**, a **16-byte GCM Authentication Tag**, and the encrypted node body (35 bytes fixed overhead per node).

Full Header & Descriptor AAD Binding: Both the 32-byte UniHeader and the 7-byte node descriptor (39 bytes total) are passed as Associated Authenticated Data (AAD) into AES-256-GCM. Any tampering with PROTO_FLAGS, TIMESTAMP_US, or node ordering immediately invalidates the GCM authentication tag.

Type ID	Identifier	Structure & Operational Purpose
0x01	NOTARY_ANCHOR	**System Node 0 (Binary Digital Notary):** Fixed 141-byte binary structure (<32s32s64sQIB) storing the 32-byte SHA-256 Data Merkle Root, the author's 32-byte Ed25519 public key, the 64-byte Ed25519 signature, creation timestamp T_0 (uint64), mandatory cooling-off interval \Delta t (uint32), and status byte (uint8).
0x02	SEMANTIC_TENSOR	**Vector Embedding Core:** Packed Little-Endian array of bit-exact IEEE 754 float32 numbers (512D/1024D) representing document or 3D scene semantics for zero-parsing cosine distance clustering.
0x03	KNOWLEDGE_GRAPH	**Structured Knowledge Graph & Deltas:** Array of atomic triplets {"s": Subject, "p": Predicate, "o": Object, "weight": float, "ego_alignment": float, "solution_link": str, "is_delta_conflict": bool}. Preserves conflicting facts in superposition (is_delta_conflict = true) until resolved.
0x04	TELEMETRY_MATRIX	**Physical, Medical & Structural Telemetry:** Compact length-prefixed pipe-delimited metric label header followed by a binary IEEE 754 float32 array (used in

Type ID	Identifier	Structure & Operational Purpose
		MZM-Pod screening, skyscraper wind-sway gyros, Venturi/HVAC thermal sensors, and coaxial Eye-Tracking).
0x05	BEHAVIOR_CONTRACT	**Dynamic Lifecycle & Hardware Rules:** Declarative parameters defining memory volatility timers (ttl_sec), target memory zone (0..3), HVAC thermal valve states (Solar_Shield / Thermal_Trap), and UI intent tags.
0x06	GENESIS_TEXT	**Human-Readable Fallback Layer:** Clean Markdown/UTF-8 text representation for legacy text editors and deterministic conversion into clinical/legal documents (DICOM, HL7 FHIR, PDF).
0x07	HOLO_WAVEFRONT	**Holographic Phase & Voxel Array:** Packed 20-byte binary records (<fffffHH: x, y, z in float32, Pockels electro-optic 2\pi phase slope in float32, rgb565 color in uint16, and laser femtosecond pulse power in uint16) for inertia-free Pockels crystal and MEMS deflection in Holo-Frame / QLOT systems.

4. Cryptographic Notary, \Delta t Timelock, and Duress Steganography

To replace bureaucratic notarization while safeguarding human agency, Node 0 (NOTARY_ANCHOR) implements three cryptographic mechanisms:

1. **Two-Phase Cooling-Off Timelock (\Delta t Validation):** For high-stakes legal, financial, architectural media, or medical consents, cooling_off_delta_t_sec specifies a mandatory time gradient \Delta t > 0 (e.g., 43,200 seconds / 12 hours). Initial signing places the container in PENDING_DELTA_T_CONFIRMATION (status = 1). Finalization requires a second cryptographic signature after T_0 + \Delta t, preventing impulsive decisions made under acute emotional affect (\Delta t < \text{minutes}).
2. **Dual-Key Duress Steganography (Primary vs. Duress Ed25519 Keys):** The user's sovereign key vault (Zone 0) maintains two distinct Ed25519 keypairs: a **Primary Keypair** (unlocked via normal authentication) and a **Duress Keypair** (unlocked via a coercion

PIN/gesture).
    - If coerced by an attacker, the user signs with the Duress Key.
    - **External Indistinguishability:** Standard public signature verification checks mathematical validity against the signer public key embedded in Node 0 and returns True (VALID). The coercer sees no warning flags on screen.
    - **Silent Quarantine Routing:** When the container reaches the user's personal cloud node, syndicate registry, or Holo-Frame controller that recognizes the Duress Public Key, the node silently routes the container into **Zone 3 (Quarantine)** and blocks execution.
3. **Unidirectional Optical Transport & FEC:** For air-gapped hardware (MZM-Pod) and skyscraper optical broadcast buses (Holo-Frame), the .uni container is appended with Reed-Solomon RS(255, 223) forward error correction parity blocks (PROTO_FLAGS & 0x0002), enabling 100% single-pass recovery over a unidirectional optical data diode without ACK/NACK feedback storms.

5. Four-Zone Dynamic Memory Triage & Semantic Compression

Host systems managing .uni files segregate storage into four logical and cryptographic memory zones:

- **Zone 0 — Sovereign Vault (The Black-Box):** Strictly local, offline-capable enclave storing root keyrings (Primary and Duress seeds), baseline user profile vectors (V_0), physical window spatial frustum coordinates (Grid_X, Grid_Y, Z_Height), and the evolution audit log (Delta_Audit_Code). Inaccessible to external networks.
- **Zone 1 — Active Stream:** Volatile working memory (RAM/SRAM) protected by ephemeral session keys (Dead Man's Switch). Upon session timeout or hardware tamper detection (e.g., perimeter frame capacitive mesh trigger), Zone 1 keys are destroyed or SRAM power is shorted to ground (Crowbar Wipe).
- **Zone 2 — Semantic Archive (The Graph):** Long-term repository of .uni containers and static 3D scene assets.
    - *Background Semantic Merging:* During idle/charging states (Asymmetric Triage), the local AI computes the Cosine Distance between SEMANTIC_TENSOR nodes across Zone 2: \text{CosineDistance}(\mathbf{A}, \mathbf{B}) = 1 - \frac{\mathbf{A} \cdot \mathbf{B}}{\Vert{}\mathbf{A}\Vert{} \Vert{}\mathbf{B}\Vert{}}
    - When \text{CosineDistance} < 0.15, candidate files are merged: duplicate triplets in KNOWLEDGE_GRAPH are deduplicated, contradictory objects for identical [Subject, Predicate] pairs are flagged as is_delta_conflict = True (Superposition), and donor file hashes are recorded in the genealogy tree before archiving originals to cold storage.
- **Zone 3 — Quarantine (The Honeypot):** Isolated sandbox for unverified external files, anomalous instantaneous ego-shifts (\Delta t < \text{minutes}), and containers signed with a Duress Key. Purged automatically upon session termination.

---

2. Полный мастер-файл `reference/python/uni_core_sdk.py` (v8.1)

Здесь объединено абсолютно всё: исправленное крипто-ядро, строгая

проверка графов знаний из v8.0[span_8](start_span)[span_8](end_span), класс `Zone2SemanticTriage` из v8.0[span_9](start_span)[span_9](end_span), новый голографический узел `0x07 HOLO_WAVEFRONT`[span_10](start_span)[span_10](end_span)[span_11](start_span)[span_11](end_span) и полный набор из **5 проверочных тестов** в конце файла.

```python
#!/usr/bin/env python3
"""
.uni Format Reference SDK (Version 8.1 Universal Reality-Verified & Holo-Stream Ready)
License: USPL-1.0 / CC-BY-4.0

Implements:
  - 32-byte O(1) Little-Endian UniHeader validation with Payload DoS protection & RS-FEC support
  - Full 39-byte (UniHeader + NodeDescriptor) AEAD AES-256-GCM AAD cryptographic binding
  - Zero-Execution TLV binary serialization across 7 universal node types (0x01..0x07)
  - Fixed 141-byte binary NOTARY_ANCHOR (0x01) & 20-byte/voxel HOLO_WAVEFRONT (0x07)
  - RFC 5869 HKDF-SHA256 per-node key derivation & 256-bit SHA-256 Merkle Tree
  - Ed25519-anchored Selective Single-Node Disclosure (Merkle Branch Proof + Notary Verification)
  - Digital Notary Anchor (Delta-T cooling-off timelock & Steganographic Duress Key routing)
  - Zone 2 Semantic Triage (Cosine Distance clustering & Knowledge Graph Delta Superposition)
"""

import struct
import hashlib
import hmac
import json
import math
import time
import os
from enum import IntEnum, IntFlag
from typing import Dict, List, Tuple, Any, Optional

# Attempt to load hardware-accelerated primitives from standard 'cryptography' package
try:
    from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

```
    from cryptography.hazmat.primitives.asymmetric.ed25519 import (
        Ed25519PrivateKey,
        Ed25519PublicKey,
    )
    HAS_CRYPTO_LIB = True
except ImportError:
    HAS_CRYPTO_LIB = False


#
# ============================================================================
# 1. UNIVERSAL NODE TYPES, FLAGS & MEMORY TRIAGE ZONES (.UNI V8.1)
#
# ============================================================================

class ProtoFlags(IntFlag):
    VERSION_8_1               = 0x0810
    TAMPER_MESH_INTACT        = 0x0001  # Hardware tamper-mesh intact
    RS_FEC_APPENDED           = 0x0002  # Reed-Solomon RS(255,223)
parity blocks appended
    STREAM_DELTA_FRAME        = 0x0004  # High-speed 120Hz Delta-Frame
(skips Node 0 Ed25519)
    SPATIAL_FRUSTUM_BROADCAST = 0x0008  # Facade-wide broadcast (slice
by Zone 0 coordinates)


class UniNodeType(IntEnum):
    NOTARY_ANCHOR     = 0x01  # System Node 0: Fixed 141B binary
Notary (Merkle + Ed25519 + Delta-T)
    SEMANTIC_TENSOR   = 0x02  # Bit-exact IEEE 754 float32 embedding
array for Zero-Parsing inference
    KNOWLEDGE_GRAPH   = 0x03  # Atomic triplets [S -> P -> O] with Ego
Alignment & Delta conflicts
    TELEMETRY_MATRIX  = 0x04  # Hardware/Medical/HVAC/Gyro sensor
telemetry (MZM-Pod, Holo-Frame)
    BEHAVIOR_CONTRACT = 0x05  # Lifecycle rules (TTL, target zone,
HVAC mode, UI intent tags)
    GENESIS_TEXT      = 0x06  # Human-readable Markdown/UTF-8 fallback
& legacy export layer
    HOLO_WAVEFRONT    = 0x07  # Binary Pockels 2*pi phase sawtooth &
MEMS 3D voxel vectors


class MemoryZone(IntEnum):
    ZONE_0_SOVEREIGN_VAULT = 0  # Strictly local root key enclave, V0
log & spatial frustum coords
    ZONE_1_ACTIVE_STREAM   = 1  # Volatile working RAM/SRAM buffer
with Dead Man's Switch
```

```
    ZONE_2_SEMANTIC_GRAPH  = 2  # Long-term vector & graph archive
(Cosine merging < 0.15)
    ZONE_3_QUARANTINE      = 3  # Isolated sandbox for untrusted
inputs & Duress alerts


#
======================================================================
# 2. CRYPTOGRAPHIC CORE (RFC 5869 HKDF, AEAD AES-256-GCM, ED25519)
#
======================================================================

def hkdf_sha256(ikm: bytes, salt: bytes, info: bytes, length: int =
32) -> bytes:
    """Derives isolated cryptographic subkeys pursuant to RFC 5869
HKDF-SHA256."""
    prk = hmac.new(salt, ikm, hashlib.sha256).digest()
    okm, prev = b"", b""
    for counter in range(1, (length // 32) + 2):
        prev = hmac.new(prk, prev + info + struct.pack("B", counter),
hashlib.sha256).digest()
        okm += prev
    return okm[:length]


def aead_encrypt(key: bytes, iv: bytes, plaintext: bytes, aad: bytes)
-> Tuple[bytes, bytes]:
    """Encrypts a node payload using AES-256-GCM (or stdlib
CTR+HMAC-SHA256 fallback)."""
    if HAS_CRYPTO_LIB:
        ct_tag = AESGCM(key).encrypt(iv, plaintext, aad)
        return ct_tag[:-16], ct_tag[-16:]
    enc_k = hmac.new(key, b"ENC" + iv, hashlib.sha256).digest()
    mac_k = hmac.new(key, b"MAC" + iv, hashlib.sha256).digest()
    ct = bytearray(len(plaintext))
    for i in range(0, len(plaintext), 32):
        ks = hashlib.sha256(enc_k + struct.pack("<I", i //
32)).digest()
        for j, b in enumerate(plaintext[i:i + 32]):
            ct[i + j] = b ^ ks[j]
    tag = hmac.new(mac_k, aad + iv + bytes(ct),
hashlib.sha256).digest()[:16]
    return bytes(ct), tag


def aead_decrypt(key: bytes, iv: bytes, ciphertext: bytes, tag: bytes,
aad: bytes) -> bytes:
    """Verifies the 128-bit authentication tag (bound to Header +
```

```
NodeDesc) and decrypts."""
    if HAS_CRYPTO_LIB:
        return AESGCM(key).decrypt(iv, ciphertext + tag, aad)
    enc_k = hmac.new(key, b"ENC" + iv, hashlib.sha256).digest()
    mac_k = hmac.new(key, b"MAC" + iv, hashlib.sha256).digest()
    exp_tag = hmac.new(mac_k, aad + iv + ciphertext,
hashlib.sha256).digest()[:16]
    if not hmac.compare_digest(tag, exp_tag):
        raise ValueError("CRITICAL SECURITY ALERT: AEAD Tag mismatch!
Node or header tampered.")
    pt = bytearray(len(ciphertext))
    for i in range(0, len(ciphertext), 32):
        ks = hashlib.sha256(enc_k + struct.pack("<I", i //
32)).digest()
        for j, b in enumerate(ciphertext[i:i + 32]):
            pt[i + j] = b ^ ks[j]
    return bytes(pt)


class SovereignKeyring:
    """
    Zone 0 Key Management Enclave.
    Maintains both a Primary Signing Key and a Steganographic Duress
Key.
    Both keys produce mathematically valid external signatures to
protect the human host,
    while trusted nodes silently route Duress-signed containers into
Zone 3 Quarantine.
    """
    def __init__(self, master_seed: bytes):
        self.master_encryption_secret = hkdf_sha256(master_seed,
b"Z0_SALT", b"ENC_MASTER")
        self._primary_seed = hkdf_sha256(master_seed, b"Z0_SALT",
b"ED25519_PRIMARY")
        self._duress_seed  = hkdf_sha256(master_seed, b"Z0_SALT",
b"ED25519_DURESS")

        if HAS_CRYPTO_LIB:
            self._primary_sk =
Ed25519PrivateKey.from_private_bytes(self._primary_seed)
            self._duress_sk  =
Ed25519PrivateKey.from_private_bytes(self._duress_seed)
            self.primary_pubkey =
self._primary_sk.public_key().public_bytes_raw()
            self.duress_pubkey  =
self._duress_sk.public_key().public_bytes_raw()
        else:
            self.primary_pubkey = hashlib.sha256(b"PUB:" +
```

```
self._primary_seed).digest()
            self.duress_pubkey  = hashlib.sha256(b"PUB:" +
self._duress_seed).digest()

    def sign_data(self, message: bytes, under_duress: bool = False) ->
Tuple[bytes, bytes]:
        """Returns (signer_pubkey_32B, signature_64B)."""
        if HAS_CRYPTO_LIB:
            sk = self._duress_sk if under_duress else self._primary_sk
            pub = self.duress_pubkey if under_duress else
self.primary_pubkey
            return pub, sk.sign(message)
        priv = self._duress_seed if under_duress else
self._primary_seed
        pub = self.duress_pubkey if under_duress else
self.primary_pubkey
        sig_p1 = hmac.new(priv, message, hashlib.sha256).digest()
        sig_p2 = hashlib.sha256(pub + message + sig_p1).digest()
        return pub, sig_p1 + sig_p2

    @staticmethod
    def verify_signature(pubkey: bytes, message: bytes, signature:
bytes) -> bool:
        """Public mathematical signature verification (returns True
for both Primary & Duress)."""
        if len(signature) != 64 or len(pubkey) != 32:
            return False
        if HAS_CRYPTO_LIB:
            try:

Ed25519PublicKey.from_public_bytes(pubkey).verify(signature, message)
                return True
            except Exception:
                return False
        exp_p2 = hashlib.sha256(pubkey + message +
signature[:32]).digest()
        return hmac.compare_digest(signature[32:], exp_p2)


#
================================================================
# 3. ZERO-EXECUTION BINARY TLV SERIALIZER (STERILE INGEST)
#
================================================================

class UniNodePayload:
    """Strictly typed binary serializer prohibiting executable code
constructs."""
```

```
    NOTARY_BIN_FMT = "<32s32s64sQIB"  # Fixed 141-byte binary
structure for Node 0
    HOLO_VOXEL_FMT = "<ffffHH"        # Fixed 20-byte binary record
per holographic beam/voxel

    @classmethod
    def pack_node(cls, node_type: UniNodeType, data: Any) -> bytes:
        if node_type == UniNodeType.NOTARY_ANCHOR:
            status_byte = 1 if int(data.get("cooling_off_delta_t_sec",
0)) > 0 else 0
            return struct.pack(
                cls.NOTARY_BIN_FMT,
                bytes.fromhex(data["data_merkle_root_256"]),
                bytes.fromhex(data["signer_pubkey_ed25519"]),
                bytes.fromhex(data["signature_ed25519"]),
                int(data["timestamp_us"]),
                int(data["cooling_off_delta_t_sec"]),
                status_byte,
            )

        elif node_type == UniNodeType.SEMANTIC_TENSOR:
            floats = [float(x) for x in data]
            return struct.pack(f"<{len(floats)}f", *floats)

        elif node_type == UniNodeType.KNOWLEDGE_GRAPH:
            triplets = []
            for item in data:
                triplets.append({
                    "s": str(item["s"]),
                    "p": str(item["p"]),
                    "o": str(item["o"]),
                    "weight": round(float(item.get("weight", 1.0)),
4),
                    "ego_alignment":
round(float(item.get("ego_alignment", 0.0)), 4),
                    "solution_link": str(item.get("solution_link",
"")),
                    "is_delta_conflict":
bool(item.get("is_delta_conflict", False)),
                })
            return json.dumps(triplets, ensure_ascii=False,
separators=(",", ":")).encode("utf-8")

        elif node_type == UniNodeType.TELEMETRY_MATRIX:
            keys = list(data.keys())
            vals = [float(data[k]) for k in keys]
            keys_hdr = "|".join(keys).encode("utf-8")
            return struct.pack("<H", len(keys_hdr)) + keys_hdr +
```

```
struct.pack(f"<{len(vals)}f", *vals)

        elif node_type == UniNodeType.BEHAVIOR_CONTRACT:
            return json.dumps(data, ensure_ascii=False,
separators=(",", ":")).encode("utf-8")

        elif node_type == UniNodeType.GENESIS_TEXT:
            return str(data).encode("utf-8")

        elif node_type == UniNodeType.HOLO_WAVEFRONT:
            buf = bytearray()
            for v in data:
                buf.extend(struct.pack(
                    cls.HOLO_VOXEL_FMT,
                    float(v["x"]),
                    float(v["y"]),
                    float(v["z"]),
                    float(v["phase_2pi"]),
                    int(v["rgb565"]),
                    int(v["pulse_mw"]),
                ))
            return bytes(buf)

        raise ValueError(f"Unsupported UniNodeType: {node_type}")

    @classmethod
    def unpack_node(cls, node_type: UniNodeType, raw_bytes: bytes) ->
Any:
        if node_type == UniNodeType.NOTARY_ANCHOR:
            root, pub, sig, ts_us, delta_t, st =
struct.unpack(cls.NOTARY_BIN_FMT, raw_bytes)
            return {
                "data_merkle_root_256": root.hex(),
                "signer_pubkey_ed25519": pub.hex(),
                "signature_ed25519": sig.hex(),
                "timestamp_us": ts_us,
                "cooling_off_delta_t_sec": delta_t,
                "status": "PENDING_DELTA_T_CONFIRMATION" if st == 1
else "FINALIZED",
            }

        elif node_type == UniNodeType.SEMANTIC_TENSOR:
            count = len(raw_bytes) // 4
            return list(struct.unpack(f"<{count}f", raw_bytes))

        elif node_type in (UniNodeType.KNOWLEDGE_GRAPH,
UniNodeType.BEHAVIOR_CONTRACT):
            return json.loads(raw_bytes.decode("utf-8"))
```

```
        elif node_type == UniNodeType.TELEMETRY_MATRIX:
            k_len = struct.unpack("<H", raw_bytes[:2])[0]
            keys = raw_bytes[2:2 + k_len].decode("utf-8").split("|")
            vals = struct.unpack(f"<{len(keys)}f", raw_bytes[2 +
k_len:])
            return {k: round(v, 4) for k, v in zip(keys, vals)}

        elif node_type == UniNodeType.GENESIS_TEXT:
            return raw_bytes.decode("utf-8")

        elif node_type == UniNodeType.HOLO_WAVEFRONT:
            voxels = []
            step = struct.calcsize(cls.HOLO_VOXEL_FMT)
            for offset in range(0, len(raw_bytes), step):
                x, y, z, ph, rgb, mw =
struct.unpack(cls.HOLO_VOXEL_FMT, raw_bytes[offset:offset + step])
                voxels.append({
                    "x": round(x, 4),
                    "y": round(y, 4),
                    "z": round(z, 4),
                    "phase_2pi": round(ph, 6),
                    "rgb565": rgb,
                    "pulse_mw": mw,
                })
            return voxels

        raise ValueError(f"Unknown UniNodeType during unpacking:
{node_type}")


#
# ============================================================================
# 4. UNIVERSAL .UNI CONTAINER ENGINE (O(1) GATE, MERKLE, SELECTIVE
EXPORT)
#
# ============================================================================

class UniUniversalContainer:
    """
    32-Byte Little-Endian UniHeader Layout ('<4sHHIIQ4s4s'):
      00..03 (4B) MAGIC_BYTES   : b'UNI8'
      04..05 (2B) PROTO_FLAGS   : Version (0x0810) & mode bitmask
      06..07 (2B) NODE_BITMAP   : Active node bitmask (nodes 0..15)
      08..11 (4B) PAYLOAD_LIMIT : Exact encrypted node stream size in
bytes
      12..15 (4B) SESSION_SALT  : 32-bit TRNG cryptographic salt
      16..23 (8B) TIMESTAMP_US  : 64-bit UTC microsecond counter
```

```
      24..27 (4B) MERKLE_TAG    : Truncated 4-byte Data Merkle root
anchor
      28..31 (4B) HEADER_ECC    : 4-byte Blake2s-32 header integrity
syndrome
    """
    HEADER_FMT = "<4sHHIIQ4s4s"
    HEADER_SIZE = 32
    MAGIC = b"UNI8"
    NODE_OVERHEAD = 35  # 7B descriptor + 12B IV + 16B GCM Tag

    @staticmethod
    def leaf_hash(node_id: int, node_type: int, raw_bytes: bytes) ->
bytes:
        return hashlib.sha256(b"UNI8_LEAF:" + struct.pack("<HB",
node_id, node_type) + raw_bytes).digest()

    @staticmethod
    def branch_hash(left: bytes, right: bytes) -> bytes:
        return hashlib.sha256(b"UNI8_BRANCH:" + left + right).digest()

    @classmethod
    def build_merkle_tree(
        cls, node_ids: List[int], leaves: List[bytes]
    ) -> Tuple[bytes, Dict[int, List[Dict[str, str]]]]:
        """Builds a SHA-256 Merkle Tree, returning the 32-byte root
and per-node Branch Proofs."""
        if not leaves:
            return b"\x00" * 32, {}
        proofs: Dict[int, List[Dict[str, str]]] = {nid: [] for nid in
node_ids}
        positions = {nid: idx for idx, nid in enumerate(node_ids)}
        level = list(leaves)

        while len(level) > 1:
            if len(level) % 2 == 1:
                level.append(level[-1])
            for nid, pos in positions.items():
                sib = level[pos + 1] if pos % 2 == 0 else level[pos -
1]
                side = "right" if pos % 2 == 0 else "left"
                proofs[nid].append({"side": side, "hash": sib.hex()})
                positions[nid] = pos // 2
            level = [cls.branch_hash(level[i], level[i + 1]) for i in
range(0, len(level), 2)]

        return level[0], proofs

    @classmethod
```

```
    def verify_header_o1(cls, buffer: bytes) -> bool:
        """Constant-time O(1) validation of the 32-byte header and
payload size limit."""
        if len(buffer) < cls.HEADER_SIZE:
            return False
        magic, flags, bitmap, payload_limit, salt, ts_us, m_tag, ecc =
struct.unpack(
            cls.HEADER_FMT, buffer[:cls.HEADER_SIZE]
        )
        if magic != cls.MAGIC:
            return False
        actual_rem = len(buffer) - cls.HEADER_SIZE
        if (flags & ProtoFlags.RS_FEC_APPENDED) and actual_rem >=
payload_limit:
            pass
        elif actual_rem != payload_limit:
            return False  # Immediate hardware/buffer drop on payload
length mismatch
        expected_ecc = hashlib.blake2s(buffer[:28],
digest_size=4).digest()
        return hmac.compare_digest(ecc, expected_ecc)

    @classmethod
    def create_container(
        cls,
        nodes_input: Dict[int, Tuple[UniNodeType, Any]],
        keyring: SovereignKeyring,
        cooling_off_seconds: int = 0,
        under_duress: bool = False,
        is_delta_frame: bool = False,
        spatial_broadcast: bool = False,
        rs_fec_tail: bytes = b"",
    ) -> bytes:
        """Assembles an encrypted .uni container with full
Header+NodeDesc AAD binding."""
        if 0 in nodes_input:
            raise ValueError("Node ID 0 is reserved for the system
NOTARY_ANCHOR node.")
        for nid in nodes_input:
            if not (1 <= nid <= 15):
                raise ValueError(f"Node ID {nid} exceeds 16-bit
NODE_BITMAP bounds (1..15).")

        session_salt = struct.unpack("<I", os.urandom(4))[0]
        timestamp_us = int(time.time() * 1_000_000)

        sorted_user_ids = sorted(nodes_input.keys())
        packed_nodes: Dict[int, Tuple[UniNodeType, bytes]] = {}
```

```
        user_leaves: List[bytes] = []

        for nid in sorted_user_ids:
            ntype, content = nodes_input[nid]
            raw_b = UniNodePayload.pack_node(ntype, content)
            packed_nodes[nid] = (ntype, raw_b)
            user_leaves.append(cls.leaf_hash(nid, int(ntype), raw_b))

        data_merkle_root, _ = cls.build_merkle_tree(sorted_user_ids,
user_leaves)

        flags = ProtoFlags.VERSION_8_1 | ProtoFlags.TAMPER_MESH_INTACT
        if spatial_broadcast:
            flags |= ProtoFlags.SPATIAL_FRUSTUM_BROADCAST
        if rs_fec_tail:
            flags |= ProtoFlags.RS_FEC_APPENDED

        if is_delta_frame:
            flags |= ProtoFlags.STREAM_DELTA_FRAME
            all_ids = sorted_user_ids
        else:
            notary_msg = data_merkle_root + struct.pack("<QI",
timestamp_us, cooling_off_seconds)
            signer_pubkey, signature = keyring.sign_data(notary_msg,
under_duress=under_duress)
            notary_dict = {
                "data_merkle_root_256": data_merkle_root.hex(),
                "signer_pubkey_ed25519": signer_pubkey.hex(),
                "signature_ed25519": signature.hex(),
                "timestamp_us": timestamp_us,
                "cooling_off_delta_t_sec": cooling_off_seconds,
            }
            notary_raw_b =
UniNodePayload.pack_node(UniNodeType.NOTARY_ANCHOR, notary_dict)
            packed_nodes[0] = (UniNodeType.NOTARY_ANCHOR,
notary_raw_b)
            all_ids = [0] + sorted_user_ids

        bitmap = sum(1 << nid for nid in all_ids)
        payload_limit = sum(cls.NODE_OVERHEAD +
len(packed_nodes[nid][1]) for nid in all_ids)

        header_28b = struct.pack(
            "<4sHHIIQ4s",
            cls.MAGIC,
            int(flags),
            bitmap,
            payload_limit,
```

```
            session_salt,
            timestamp_us,
            data_merkle_root[:4],
        )
        ecc = hashlib.blake2s(header_28b, digest_size=4).digest()
        full_header_32b = header_28b + ecc

        salt_bytes = struct.pack("<I", session_salt)
        encrypted_chunks: List[bytes] = []

        for nid in all_ids:
            ntype, raw_b = packed_nodes[nid]
            node_desc = struct.pack("<HBI", nid, int(ntype),
len(raw_b))
            iv = hashlib.sha256(salt_bytes + struct.pack("<H", nid) +
os.urandom(8)).digest()[:12]
            node_key = hkdf_sha256(keyring.master_encryption_secret,
salt_bytes, f"NODE_{nid}".encode())
            ct, tag = aead_encrypt(node_key, iv, raw_b,
aad=full_header_32b + node_desc)
            encrypted_chunks.append(node_desc + iv + tag + ct)

        return full_header_32b + b"".join(encrypted_chunks) +
rs_fec_tail

    @classmethod
    def inspect_and_route(cls, uni_binary: bytes, keyring:
SovereignKeyring) -> Dict[str, Any]:
        """Decrypts container, verifies Notary signature (or
Delta-Frame AAD), and routes to Memory Zone."""
        if not cls.verify_header_o1(uni_binary):
            raise ValueError("O(1) Hardware Gate: Invalid header or
payload length mismatch!")

        full_header_32b = uni_binary[:cls.HEADER_SIZE]
        magic, flags, bitmap, payload_limit, session_salt, ts_us,
m_tag, ecc = struct.unpack(
            cls.HEADER_FMT, full_header_32b
        )
        salt_bytes = struct.pack("<I", session_salt)

        stream_end = cls.HEADER_SIZE + payload_limit
        offset = cls.HEADER_SIZE
        unpacked_nodes: Dict[int, Dict[str, Any]] = {}

        while offset < stream_end:
            node_desc = uni_binary[offset:offset + 7]
            nid, ntype_int, raw_len = struct.unpack("<HBI", node_desc)
```

```
            iv = uni_binary[offset + 7:offset + 19]
            tag = uni_binary[offset + 19:offset + 35]
            ct = uni_binary[offset + 35:offset + 35 + raw_len]
            offset += 35 + raw_len

            node_key = hkdf_sha256(keyring.master_encryption_secret,
salt_bytes, f"NODE_{nid}".encode())
            raw_b = aead_decrypt(node_key, iv, ct, tag,
aad=full_header_32b + node_desc)
            ntype = UniNodeType(ntype_int)
            unpacked_nodes[nid] = {
                "type": ntype.name,
                "data": UniNodePayload.unpack_node(ntype, raw_b),
            }

        if flags & ProtoFlags.STREAM_DELTA_FRAME:
            return {
                "frame_mode": "FAST_DELTA_FRAME_120HZ",
                "public_signature_valid": True,
                "assigned_memory_zone":
MemoryZone.ZONE_1_ACTIVE_STREAM.name,
                "internal_security_verdict": "VERIFIED_SESSION_AEAD:
Header-bound GCM Delta Frame.",
                "nodes": unpacked_nodes,
            }

        notary = unpacked_nodes[0]["data"]
        signer_pub = bytes.fromhex(notary["signer_pubkey_ed25519"])
        signature = bytes.fromhex(notary["signature_ed25519"])
        data_root = bytes.fromhex(notary["data_merkle_root_256"])
        delta_t = notary["cooling_off_delta_t_sec"]

        signed_msg = data_root + struct.pack("<QI",
notary["timestamp_us"], delta_t)
        is_sig_valid = SovereignKeyring.verify_signature(signer_pub,
signed_msg, signature)

        if hmac.compare_digest(signer_pub, keyring.duress_pubkey):
            zone = MemoryZone.ZONE_3_QUARANTINE
            verdict = "SILENT_DURESS_ALERT: Coercion key detected!
Isolated in Zone 3 Quarantine."
        elif not is_sig_valid:
            zone = MemoryZone.ZONE_3_QUARANTINE
            verdict = "INVALID_SIGNATURE: Cryptographic verification
failed."
        else:
            zone = MemoryZone.ZONE_1_ACTIVE_STREAM
            verdict = "VERIFIED_SOVEREIGN: Authentic Primary
```

```
Signature."

        return {
            "frame_mode": "KEYFRAME_NOTARIZED",
            "public_signature_valid": is_sig_valid,
            "assigned_memory_zone": zone.name,
            "internal_security_verdict": verdict,
            "nodes": unpacked_nodes,
        }

    @classmethod
    def export_selective_slice(
        cls, uni_binary: bytes, keyring: SovereignKeyring,
target_node_id: int
    ) -> Dict[str, Any]:
        """Exports a single decrypted user node accompanied by its
Merkle Branch Proof & Node 0 Notary Seal."""
        if target_node_id == 0:
            raise ValueError("Node 0 is the Notary Anchor; specify a
user node (1..15) to disclose.")
        if not cls.verify_header_o1(uni_binary):
            raise ValueError("Container failed O(1) header check.")

        full_header_32b = uni_binary[:cls.HEADER_SIZE]
        _, flags, bitmap, payload_limit, session_salt, _, _, _ =
struct.unpack(
            cls.HEADER_FMT, full_header_32b
        )
        if not (bitmap & (1 << target_node_id)):
            raise ValueError(f"Node {target_node_id} is not present in
container.")

        salt_bytes = struct.pack("<I", session_salt)
        offset = cls.HEADER_SIZE
        stream_end = cls.HEADER_SIZE + payload_limit
        user_ids, user_leaves = [], []
        target_raw, target_type = b"", UniNodeType.GENESIS_TEXT
        notary_data: Dict[str, Any] = {}

        while offset < stream_end:
            node_desc = uni_binary[offset:offset + 7]
            nid, ntype_int, raw_len = struct.unpack("<HBI", node_desc)
            iv = uni_binary[offset + 7:offset + 19]
            tag = uni_binary[offset + 19:offset + 35]
            ct = uni_binary[offset + 35:offset + 35 + raw_len]
            offset += 35 + raw_len

            node_key = hkdf_sha256(keyring.master_encryption_secret,
```

```
salt_bytes, f"NODE_{nid}".encode())
            raw_b = aead_decrypt(node_key, iv, ct, tag,
aad=full_header_32b + node_desc)
            if nid == 0:
                notary_data =
UniNodePayload.unpack_node(UniNodeType.NOTARY_ANCHOR, raw_b)
            else:
                user_ids.append(nid)
                user_leaves.append(cls.leaf_hash(nid, ntype_int,
raw_b))
                if nid == target_node_id:
                    target_raw = raw_b
                    target_type = UniNodeType(ntype_int)

        data_root_256, proofs = cls.build_merkle_tree(user_ids,
user_leaves)
        return {
            "node_id": target_node_id,
            "node_type": int(target_type),
            "node_type_name": target_type.name,
            "decoded_data": UniNodePayload.unpack_node(target_type,
target_raw),
            "raw_payload_hex": target_raw.hex(),
            "data_merkle_root_256": data_root_256.hex(),
            "merkle_branch_proof": proofs[target_node_id],
            "notary_anchor": notary_data,
        }

    @classmethod
    def verify_selective_slice(
        cls, exported_slice: Dict[str, Any], expected_pubkey_hex:
Optional[str] = None
    ) -> bool:
        """Verifies a single exported node against the 256-bit Merkle
root AND the author's Ed25519 signature."""
        curr = cls.leaf_hash(
            exported_slice["node_id"],
            exported_slice["node_type"],
            bytes.fromhex(exported_slice["raw_payload_hex"]),
        )
        for step in exported_slice["merkle_branch_proof"]:
            sib = bytes.fromhex(step["hash"])
            curr = cls.branch_hash(curr, sib) if step["side"] ==
"right" else cls.branch_hash(sib, curr)

        notary = exported_slice["notary_anchor"]
        if not hmac.compare_digest(curr.hex(),
notary["data_merkle_root_256"]):
```

```
            return False

        if expected_pubkey_hex and not
hmac.compare_digest(notary["signer_pubkey_ed25519"],
expected_pubkey_hex):
            return False

        signed_msg = (
            bytes.fromhex(notary["data_merkle_root_256"])
            + struct.pack("<QI", notary["timestamp_us"],
notary["cooling_off_delta_t_sec"])
        )
        return SovereignKeyring.verify_signature(
            bytes.fromhex(notary["signer_pubkey_ed25519"]),
            signed_msg,
            bytes.fromhex(notary["signature_ed25519"]),
        )


#
================================================================================
# 5. ZONE 2 SEMANTIC TRIAGE (COSINE CLUSTERING & DELTA SUPERPOSITION)
#
================================================================================

class Zone2SemanticTriage:
    """Background vector clustering and Knowledge Graph merging with
Delta conflict tracking."""

    @staticmethod
    def cosine_distance(vec_a: List[float], vec_b: List[float]) ->
float:
        dot = sum(a * b for a, b in zip(vec_a, vec_b))
        norm_a = math.sqrt(sum(a * a for a in vec_a))
        norm_b = math.sqrt(sum(b * b for b in vec_b))
        if norm_a == 0.0 or norm_b == 0.0:
            return 1.0
        return round(1.0 - (dot / (norm_a * norm_b)), 6)

    @staticmethod
    def merge_knowledge_graphs(
        graph_a: List[Dict[str, Any]], graph_b: List[Dict[str, Any]]
    ) -> List[Dict[str, Any]]:
        grouped: Dict[Tuple[str, str], List[Dict[str, Any]]] = {}
        for edge in graph_a + graph_b:
            key = (edge["s"], edge["p"])
            grouped.setdefault(key, [])
            if not any(existing["o"] == edge["o"] for existing in
```

```
grouped[key]):
                grouped[key].append(dict(edge))

        merged_list = []
        for _, edges in grouped.items():
            conflict = len(edges) > 1
            for e in edges:
                e["is_delta_conflict"] = conflict
                merged_list.append(e)
        return merged_list


#
================================================================================
# SELF-TEST & VERIFICATION SUITE (ALL 5 CORE & HOLOGRAPHIC TESTS)
#
================================================================================
if __name__ == "__main__":

print("================================================================
===")
    print(" .uni Universal Reference SDK v8.1 - Complete Verification
Suite")
    print(f" Crypto Backend: {'Hardware AES-256-GCM & Ed25519' if
HAS_CRYPTO_LIB else 'Stdlib AEAD Fallback'}")

print("================================================================
===\n")

    keyring =
SovereignKeyring(master_seed=b"SOVEREIGN_ROOT_SEED_2026_V8_1")

    sample_nodes = {
        1: (UniNodeType.SEMANTIC_TENSOR, [0.82, 0.14, -0.33, 0.91]),
        2: (UniNodeType.KNOWLEDGE_GRAPH, [
            {"s": "MZM-Pod", "p": "uses_memory", "o": "Volatile_SRAM",
"weight": 0.99, "ego_alignment": 0.95},
            {"s": "Dump_Valve", "p": "triggers_at", "o": "1.2_atm",
"weight": 0.95, "ego_alignment": 0.90},
        ]),
        3: (UniNodeType.TELEMETRY_MATRIX, {"carotid_IMT_mm": 0.72,
"VAT_SAT_ratio": 0.41, "PWV_ms": 6.8}),
        4: (UniNodeType.BEHAVIOR_CONTRACT, {"ttl_sec": 3600,
"ui_intent": "[Weight: Focus]", "zone": 1}),
        5: (UniNodeType.GENESIS_TEXT, "# Session Summary\nVascular and
metabolic parameters within normal range."),
        7: (UniNodeType.HOLO_WAVEFRONT, [
            {"x": 12.5, "y": 40.0, "z": 2.5, "phase_2pi": 3.14159,
```

```
"rgb565": 0xF800, "pulse_mw": 850},
            {"x": 12.6, "y": 40.0, "z": 2.8, "phase_2pi": 1.57079,
"rgb565": 0x07E0, "pulse_mw": 820},
        ]),
    }

    # Test 1: Normal Container with 12-hour (43,200s) Delta-T
Cooling-Off Timelock
    uni_normal = UniUniversalContainer.create_container(
        sample_nodes, keyring, cooling_off_seconds=43200,
under_duress=False
    )
    inspected_normal =
UniUniversalContainer.inspect_and_route(uni_normal, keyring)
    print(f"[Test 1] Standard Container Packed ({len(uni_normal)}
bytes)")
    print(f"         Public Signature Valid :
{inspected_normal['public_signature_valid']}")
    print(f"         Assigned Memory Zone   :
{inspected_normal['assigned_memory_zone']}")
    print(f"         Notary Status          :
{inspected_normal['nodes'][0]['data']['status']}\n")

    # Test 2: Coerced Signature (Steganographic Duress Key)
    uni_duress = UniUniversalContainer.create_container(
        sample_nodes, keyring, cooling_off_seconds=0,
under_duress=True
    )
    inspected_duress =
UniUniversalContainer.inspect_and_route(uni_duress, keyring)
    print("[Test 2] Coerced Container (Signed with Duress Key)")
    print(f"         External Verifier Sees : Signature Valid =
{inspected_duress['public_signature_valid']}")
    print(f"         Sovereign Node Action  :
{inspected_duress['internal_security_verdict']}\n")

    # Test 3: Selective Single-Node Disclosure (Node 3 Telemetry) &
Ed25519-Anchored Merkle Proof
    telemetry_slice =
UniUniversalContainer.export_selective_slice(uni_normal, keyring,
target_node_id=3)
    slice_verified = UniUniversalContainer.verify_selective_slice(
        telemetry_slice,
expected_pubkey_hex=keyring.primary_pubkey.hex()
    )
    print("[Test 3] Selective Single-Node Export (Node 3:
TELEMETRY_MATRIX)")
    print(f"         Disclosed Metrics      :
```

```
{telemetry_slice['decoded_data']}")
    print(f"         Merkle + Ed25519 Proof : {slice_verified} (Nodes
1, 2, 4, 5, 7 remain unexposed)\n")

    # Test 4: Zone 2 Semantic Triage & Delta Conflict Superposition
    c_dist = Zone2SemanticTriage.cosine_distance([0.82, 0.14, -0.33,
0.91], [0.80, 0.15, -0.31, 0.89])
    merged = Zone2SemanticTriage.merge_knowledge_graphs(
        sample_nodes[2][1],
        [{"s": "Dump_Valve", "p": "triggers_at", "o": "1.5_atm",
"weight": 0.80}],
    )
    print(f"[Test 4] Zone 2 Semantic Triage | Cosine Distance =
{c_dist} (< 0.15 threshold)")
    for triplet in merged:
        delta_tag = " [DELTA CONFLICT SUPERPOSITION]" if
triplet["is_delta_conflict"] else ""
        print(f"         [{triplet['s']}] -> [{triplet['p']}] ->
[{triplet['o']}]{delta_tag}")
    print()

    # Test 5: Holo-Frame 120Hz Delta-Stream & Reed-Solomon Facade
Broadcast
    delta_pkt = UniUniversalContainer.create_container(
        {7: sample_nodes[7]},
        keyring,
        is_delta_frame=True,
        spatial_broadcast=True,
        rs_fec_tail=b"\xAA" * 32,
    )
    res_df = UniUniversalContainer.inspect_and_route(delta_pkt,
keyring)
    print(f"[Test 5] Holo-Frame 120Hz Delta Packet ({len(delta_pkt)}
bytes inc. 32B RS-FEC tail)")
    print(f"         Frame Mode             : {res_df['frame_mode']}")
    print(f"         Decoded Wavefront      :
{len(res_df['nodes'][7]['data'])} Pockels/MEMS voxels ready")
```

3. Полный мастер-файл README.md (v8.1)

```
# `.uni` (Universal Node Identity / Universal Neural Interface)
Specification v8.1

[![License:
USPL-1.0](https://img.shields.io/badge/License-USPL--1.0-blue.svg)](LI
CENSE)
[![Status: Prior Art
```

Baseline](https://img.shields.io/badge/Status-Prior%20Art%20v8.1-green.svg)](SPECIFICATION.md)

Official technical specification and reference SDK for the **`.uni` (v8.1 Universal)** non-executable semantic, telemetry, holographic wavefront, and cryptographic notary container.

The `.uni` format eliminates text-parsing overhead (`Zero-Parsing Inference`) and arbitrary code execution (`Zero-Execution Environment`) by packaging multidimensional floating-point tensors, atomic Knowledge Graph triplets, physical sensor telemetry, Pockels/MEMS holographic phase arrays, and binary `Ed25519` digital notary anchors into a single 32-byte-header binary container with full-header-bound `AES-256-GCM` encryption and selective `Merkle Branch Proof` disclosure.

Key Features (v8.1 Universal Baseline)

1. **$O(1)$ 32-Byte Hardware-Aligned Header (`UniHeader`):** Constant-time verification of packet integrity (`Blake2s-32`), 64-bit microsecond epoch timestamps, Reed-Solomon `RS(255, 223)` optical broadcast flags, and exact payload byte limits (`PAYLOAD_LIMIT`) prior to dynamic memory allocation.
2. **Seven Universal TLV Node Types:**
   * `0x01 NOTARY_ANCHOR` — Fixed 141-byte binary Notary (256-bit Merkle Root, `Ed25519` signature, Δt cooling-off timelock, and steganographic `Duress Key` support).
   * `0x02 SEMANTIC_TENSOR` — Bit-exact IEEE 754 `float32` vector embeddings for instant cosine similarity clustering.
   * `0x03 KNOWLEDGE_GRAPH` — Atomic `[Subject -> Predicate -> Object]` triplets with `ego_alignment` weights and Delta Conflict Superposition.
   * `0x04 TELEMETRY_MATRIX` — High-density binary sensor, medical (`MZM-Pod`), structural gyro, and HVAC telemetry matrices.
   * `0x05 BEHAVIOR_CONTRACT` — Volatility timers (`TTL`), memory zone routing, HVAC thermal states, and UI intent tags.
   * `0x06 GENESIS_TEXT` — Human-readable Markdown/UTF-8 fallback for legacy transcoding (DICOM, HL7 FHIR, PDF).
   * `0x07 HOLO_WAVEFRONT` — Packed 20-byte binary holographic voxel & Pockels 2π phase-gradient arrays for `Holo-Frame` / `QLOT` solid-state displays.
3. **Selective Zero-Knowledge Node Disclosure:** Derived per-node keys (`RFC 5869 HKDF-SHA256` + `AES-256-GCM` with full 39-byte Header+Descriptor AAD binding) allow the owner to decrypt and prove the authenticity of a single node to a third party via an `Ed25519`-signed SHA-256 `Merkle Branch Proof` while keeping all other

nodes concealed.
4. **4-Zone Dynamic Memory Triage:** Automated routing across `Zone 0 (Sovereign Vault & Spatial Frustum)`, `Zone 1 (Active 120Hz Stream)`, `Zone 2 (Semantic Archive)`, and `Zone 3 (Quarantine)`.

Quick Start

```bash
python3 reference/python/uni_core_sdk.py
```